

Project Deep Dive Analysis Document

AI-Powered SRE Incident Management and Diagnostics Platform

Project Technical Analysis Document

This document provides a comprehensive technical analysis of the SRE AI Incident Management Agent platform, based strictly on the current code implementation of the frontend and backend systems.

1. Executive Summary

- **What the Project Does:** The platform is an automated, AI-assisted site reliability engineering (SRE) diagnostic and escalation system. It ingests raw telemetry alerts from alert managers (e.g., Prometheus AlertManager, Grafana Loki), deduplicates them via dynamic Redis Bloom filters, groups them into single incidents using dependency-aware correlation algorithms, and runs a stateful LLM diagnostic workflow (using LangGraph and Groq Llama 3.3 70B) to generate root cause hypotheses. Finally, it provides an operator-controlled workflow for approving escalations to Slack (via interactive Block Kit cards) and compiles post-mortem reports.

- **Main Objectives:**

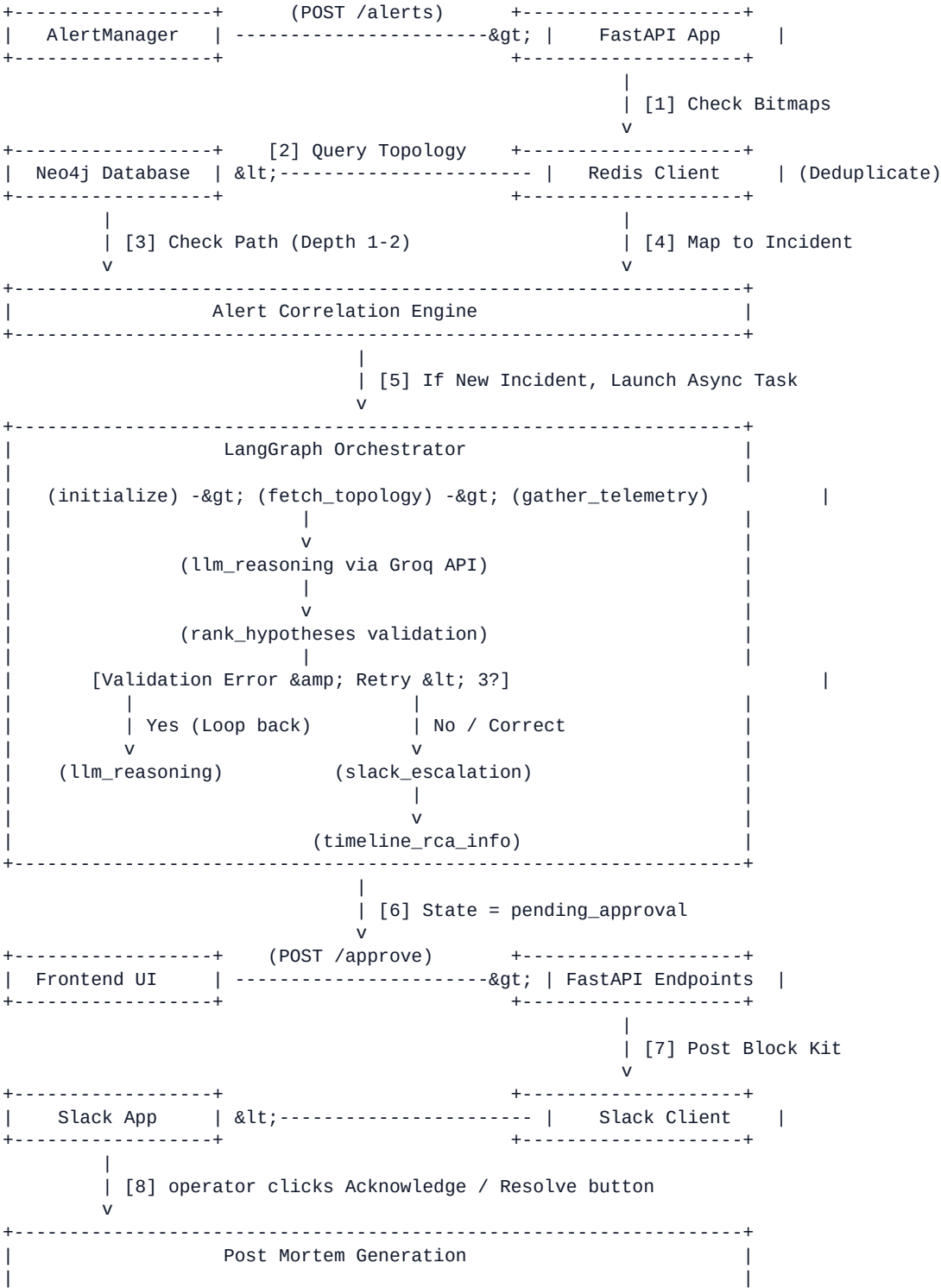
1. Reduce alert fatigue by correlating cascading alerts into single, unified incidents.
2. Minimize Mean Time To Repair (MTTR) by running automated, zero-latency root-cause diagnostics.
3. Streamline escalation operations through manual-approval loops, ensuring high-fidelity Slack dispatches and standardized incident reports.

- **Core Architecture:**

- **Frontend:** React (TypeScript), Vite, TailwindCSS (configured via CSS rules), React Router DOM (v6), Lucid Icons, and canvas-based graph rendering for topology.
- **Backend:** FastAPI (Python), uvicorn, Pydantic (data validation/schemas), OpenTelemetry (auto-instrumentation and tracing), Prometheus Client (custom metrics).
- **State & Deduplication:** Redis (bitmaps for Bloom filters, hash/set indices, and JSON key storage).
- **Topology Graph:** Neo4j database (Cypher query language for nodes/relationships).
- **AI Orchestration:** LangGraph (StateGraph state machines with checkpointers) and Groq API client (`llama-3.3-70b-versatile` in JSON mode).

2. End-to-End Architecture

The following flow illustrates how an incoming telemetry alert propagates through the backend stack, undergoes AI diagnosis, and escalates to a Slack channel:



```
| 1. State -> 'resolved' |
| 2. Compile metadata, hypotheses, and timeline |
| 3. Generate executive summary via Groq |
| 4. Save markdown file to disk (storage/rcas/{id}.md) |
| 5. Set Redis cache (rca:{id}) with 7-day TTL |
+-----+-----+
```

Flow Breakdown:

- Alert Ingestion:** Raw JSON payloads from Prometheus AlertManager or Loki are received by **POST /alerts** inside [routes.py](#).
- Deduplication layer:** The alert is hashed. Redis bitmaps are checked under rotating keys **dedup:bloom:{epoch}** (30-second windows) to see if all Bloom filter indices are set. If true, the alert is dropped as a duplicate.
- Correlation Engine:** Standardized alert is correlated against all active incidents in Redis. It checks:
 - **Time window:** If the alert timestamp falls within 300 seconds of the incident's last update.
 - **Direct match:** If the alert's service matches any service already impacted in the incident.
 - **Dependency match:** It asks Neo4j if there is a dependency path (depth 1 or 2) between the alert's service and the incident's affected services list.
- State Mapping:** If correlated, the alert is merged into the existing incident. If it bridges multiple incidents, the incidents are merged into the oldest primary incident. Otherwise, a new incident is created.
- AI Investigation Workbench (LangGraph):** If a new incident is created, an asynchronous task calls the compiled StateGraph. The workflow fetches topology upstreams/downstreams, gathers Prometheus metrics and Loki logs concurrently, calls Groq, parses the JSON structure, self-corrects up to 3 times on formatting errors, and updates the incident status in Redis to **pending_approval**.
- Escalation Approval:** Operator triggers **POST /incidents/{id}/approve** from the frontend, prompting the Slack client to build an interactive Block Kit message and dispatch it to Slack.
- Resolution and Post-Mortem:** When resolved via the API or Slack button, the **RCAGenerator** creates the Markdown document, stores it in **storage/rcas/{id}.md**, and caches it in Redis.

3. Frontend Deep Dive

The frontend is a single-page application built on Vite and React. It communicates with the FastAPI backend using a unified Axios instance defined in [client.ts](#).

Page Catalog:

Page	Purpose	Data Source	API Calls	User Actions	Expected Outputs
Login LoginPage.tsx	Operator authentication gateway.	LocalStorage.	None.	Input Name, Email, Password, submit form.	Set credentials in LocalStorage, redirect to <code>/</code> .
Overview OverviewPage.tsx	Operational dashboard of key metrics.	useIncidentStore (incidents, systemHealth).	fetchIncidents() , fetchSystemHealth()	Click active incident to open triage detail, click links.	Dynamic KPI count updates, service connection indicators, recent activity feed.
Incidents DashboardPage.tsx	Triage workbench to filter and inspect incidents.	useIncidentStore (incidents).	fetchIncidents() , resolveIncident(...)	Search by name/ID, filter by severity/state, click Resolve.	Filtered list of incidents in grid or tabular format.
RCA Analysis RCAViewerPage.tsx	AI diagnostic viewer and exporter.	useIncidentStore (resolved incidents).	fetchIncidents() , /rca/{id}/export (PDF), /rca/{id}/json (JSON)	Select incident, click PDF Export, click JSON Export.	Top 3 hypotheses, evidence, confidence, recommendations, and latency metrics.
Timeline TimelinePage.tsx	Audit trail of event timelines.	useIncidentStore (timeline list).	fetchIncidents()	Select incident, click "Inspect Triage Workbench".	Vertical chronological stream tracking alerts, AI milestones, and human operators.
Post Mortems PostMortemsPage.tsx	Archive for finalized reports.	useIncidentStore & /rca/{id} (API text).	fetchIncidents() , api.get('/rca/{id}')	Select report, download PDF or JSON, read checklists.	Split layout: Executive summary, hypotheses table, corrective actions checklist.
Services ServicesPage.tsx	Microservice registry catalog.	Static metadata list mapped to active incidents.	fetchIncidents()	View latency, CPU, memory metrics, and outages.	Health badges (healthy, warning, critical), metric trend sparklines.
Alerts AlertsPage.tsx	Raw alerts telemetry ledger.	useIncidentStore (incident alerts list).	fetchIncidents()	Filter by severity (ALL, CRITICAL, WARNING).	Table with alert name, severity, source component, and timestamp.
Escalations EscalationsPage.tsx	Control dispatcher mapping manual/AI approval escalations, Slack card routes, and action items.	useIncidentStore (incident states).	fetchIncidents()	Select incident card to view details in the workbench.	Three-column dispatcher grid: Pending, Approved, and Rejected.
Dependency Map App.tsx	Dynamic graph visualization layout.	/topology/graph endpoint.	api.get('/topology/graph') inside DependencyGraph.tsx	View nodes, drag/hover service nodes.	Canvas rendering Service and Database nodes with active status lights.

Health Dashboard AgentHealthPage.tsx	Observability health logs.	<code>useIncidentStore & /dashboard/metrics.</code>	<code>fetchSystemHealth(), api.get('/dashboard/metrics')</code>	Inspect database latencies, LLM usage.	Charts mapping total AI costs, token counts, and engine connectivity status.
--	----------------------------	---	---	--	--

4. Alert Correlation Engine

The engine is implemented in [correlation.py](#).

- **Correlation Window:** Configured using `CORRELATION_WINDOW_SECONDS = 300` (5 minutes). This is a sliding window based on the incident's `updated_at` field.

- **Service Matching:** Direct service checking:

```
if alert.service in incident.services_affected:  
    # Direct service match found
```

- **Dependency Matching:** Under the hood, it queries Neo4j via the `check_dependency_path(service_a, services_list)` method in `neo4j_client.py`. It executes the following Cypher query to check paths of length 1 or 2:

```
MATCH p = (s1:Service)-[:DEPENDS_ON*1..2]-(s2:Service)  
WHERE s1.name = $service_a AND s2.name IN $services_list  
RETURN count(p) > 0 AS connected
```

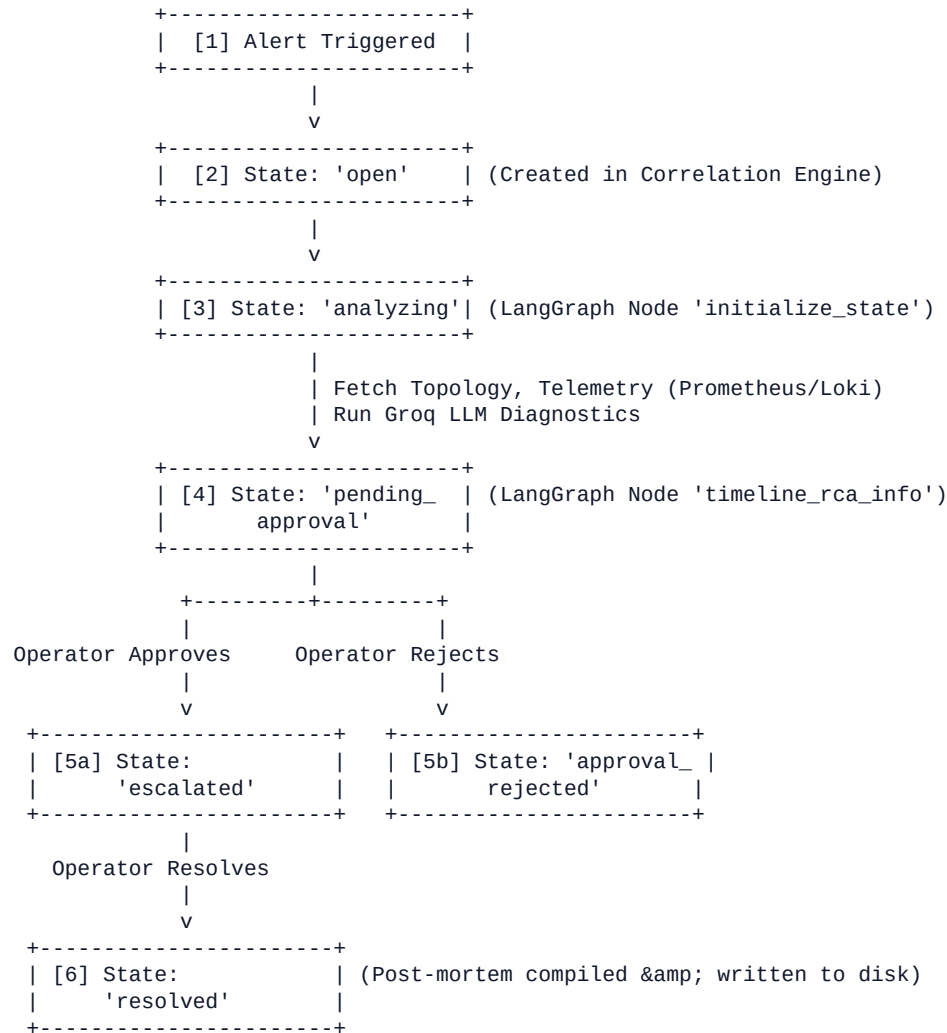
- **Incident Creation:** When no matching active incidents are found, `_create_incident(alert)` is called. It sets the status to `"open"`, logs the start time, and generates initial timeline entries.
- **Incident Merging:**
- **Single match:** Merges the alert directly into the matching incident, updates the severity if the new alert is higher (using priority ranking: `info < warning < critical`), and appends a `TimelineItem`.
- **Multiple matches:** If an incoming alert matches multiple active incidents, `_merge_multiple_incidents(incidents, alert)` groups them. It sorts incidents by creation time, keeps the oldest as the primary incident, changes the state of secondary incidents to `"merged"`, updates their `merged_into` attribute to the primary ID, and transfers all alerts and timeline records to the primary incident.

Concrete Example:

1. `payment-service` triggers `Http5xxRateHigh` at 12:00:00. This creates **Incident A**.
2. `api-gateway` triggers `GatewayTimeout` at 12:02:00. The engine queries Neo4j to see if `api-gateway` is connected to `payment-service` (depth 1-2).
3. The Cypher query returns `connected = True` because `api-gateway` depends on `payment-service` in the seeded topology.
4. The gateway alert is merged into **Incident A**, avoiding the creation of a duplicate incident.

5. Incident Lifecycle

The lifecycle follows a strict sequence of state transitions managed in Redis and LangGraph:



6. LLM Root Cause Analysis

The LangGraph workflow (defined in [workflow.py](#)) operates as a stateful worker:

- **LangGraph Workflow Structure:**

1. **initialize_state**: Fetches the incident from Redis, sets state to **"analyzing"**, and initializes context.
2. **fetch_topology**: Traverses upstreams/downstreams of affected services to expand the telemetry collection scope.
3. **gather_telemetry**: Concurrently queries Prometheus for CPU, memory, and error rates, and Loki for error/timeout logs around the alert start time (-5m to +5m).
4. **llm_reasoning**: Assembles the context blocks (Active alerts, topology, metrics, logs) and queries Groq.
5. **rank_hypotheses**: Validates the response. If formatting errors exist, it increments the retry count and loops back to **llm_reasoning**. If retries reach 3, it falls back to a safe programmatic explanation.
6. **slack_escalation**: Saves hypotheses to Redis and pauses for operator review.
7. **timeline_rca_info**: Finalizes the timeline and sets the incident state to **pending_approval**.
 - **Groq Integration**: Located in [groq_client.py](#). It interacts with the **llama-3.3-70b-versatile** model. It forces JSON output formatting via **"response_format": {"type": "json_object"}**.
 - **Prompting Flow**: The system prompt instructs Llama to behave as a Senior Staff SRE, enforcing cascade failure tracing, saturation analysis, and strict schema validation.
 - **Hypothesis Generation**: The top 3 hypotheses are generated by the model based on telemetry evidence. For each hypothesis, it outputs:
 1. **rank** (1 to 3).
 2. **hypothesis** (Proposed root cause).
 3. **confidence_score** (Float between 0.0 and 1.0).
 4. **evidence** (List of log files or metric observations).
 5. **recommended_action** (Operator mitigation checklist).

7. Service Dependency Graph

The dependency graph is managed by the Neo4j client in [neo4j_client.py](#).

- **Neo4j Schema:**
- **Nodes:**
 - `(:Service {name, language, version, updated_at})`
 - `(:Database {name, type, updated_at})`
- **Relationships:**
 - `(:Service)-[:DEPENDS_ON {protocol, p99_latency_threshold_ms, updated_at}]->(:Service)`
 - `(:Service)-[:DEPENDS_ON {protocol, p99_latency_threshold_ms, updated_at}]->(:Database)`
- **Graph Population Method:**
- **Static Topology (B):** The graph is seeded during FastAPI startup if Neo4j is empty. It reads from [baseline_topology.json](#) (which contains static configurations for `api-gateway`, `auth-service`, `payment-service`, `shipping-service`, and `inventory-service`).
- **Batch Loading:** The endpoint `POST /topology/load` accepts a custom schema payload to overwrite or seed the database.
- **Code Evidence:** Lines 60–91 of [main.py](#) check if the graph is empty and load the static JSON file:

```
topology_path = Path(__file__).resolve().parent.parent / "scripts" / "baseline_topology.json"
if topology_path.exists():
    with open(topology_path, "r") as f:
        data = json.load(f)
    # Calls create_service_node, create_database_node, create_dependency
```

There is no automatic trace parsing logic (e.g., extracting trace graphs from Jaeger or OpenTelemetry collectors in real-time) to populate the topology.

8. Slack Escalation System

The Slack escalation is implemented in [slack_client.py](#).

- **Approval Flow:** Operator review is required. When the operator clicks "Approve & Send Escalation" on the frontend, it sends a payload to `POST /incidents/{id}/approve` containing the target channel. The backend then calls the Slack Web API.
- **Backend Endpoint:** Mapped to `POST /incidents/{id}/approve` inside `routes.py`.
- **Slack SDK Integration:** Uses the Slack Bolt framework for Python:

```
from slack_bolt.async_app import AsyncApp
slack_app = AsyncApp(token=settings.SLACK_BOT_TOKEN, signing_secret=settings.SLACK_SIGNING_SECRET)
```

- **Block Kit Structure:** Built in `SlackClient` using helper methods:
- **header_block:** Displays the severity emoji (■ or ■■) and incident ID.
- **context_block:** Shows timestamps and affected services.
- **hypotheses_block:** Visualizes the ranked root causes and confidence scores using a text-based progress bar (e.g., [■■■■■■■■■■] 40%).
- **actions_block:** Appends dynamic buttons with `action_id="slack_ack_incident"` and `action_id="slack_resolve_incident"` for interactive operators.
- **Channel Routing:** The method `get_routing_channel(service_name)` queries the Redis hash key `"slack:channel_routing"`. If a service-specific mapping is found, it routes the message to that channel. Otherwise, it falls back to the default channel defined in `settings.SLACK_CHANNEL`.
- **Mock Mode vs Real Mode:** Real mode requires a valid bot token. If `settings.SLACK_BOT_TOKEN == "mock_token"`, mock mode is active. It skips the HTTP post, increments Prometheus metrics, logs the payload to the console, and returns a successful mock response (`ts="mock_ts_12345"`).

9. Timeline System

The timeline system provides an audit trail of SRE event milestones, tracing from alert ingestion to final resolution.

- **Event Generation:** Chronological events are generated by the backend and appended as **TimelineItem** models. Sources include:
 - **"prometheus"**: Alert trigger entries.
 - **"system"**: Auto-ingestion and LangGraph reasoning execution milestones.
 - **"slack_operator"**: Human approvals, acknowledgments, and resolutions.
- **Timeline Storage:** Stored as a JSON array inside the **timeline** field of the **IncidentStateModel** in Redis:

```
incident_key = f"incident:state:{incident.id}"  
# Saved via redis_manager.save_incident(incident)
```
- **Timeline Rendering:** The React component [TimelinePage.tsx](#) queries **GET /timeline/{id}**. It renders a vertical rail connected by status nodes (color-coded by event type).
- **Incident Milestones:**
 - **incident_ingested**: Logged on alert receipt.
 - **agent_milestone / "AI RCA hypotheses generated."**: Added on LangGraph completion.
 - **operator_action / "Incident escalation approved..."**: Added on approval.
 - **operator_action / "Incident marked resolved..."**: Added on resolution.

10. Post Mortem Generation

- **When Generated:** Compiles when the operator resolves the incident (via **POST /incidents/{id}/resolve** or the Slack resolve button).
- **Data Sources:** Aggregates the final sorted timeline, the highest-confidence hypotheses, the affected services list, and metadata (MTTR, timestamps).
- **RCA Integration:** Queries Groq to write a professional executive summary. If Groq is disabled or offline, it falls back to a programmatic summary:

```
summary = f"Incident {id} affected services: {services}. Primary root cause is hypothesized to be: {hyp1}..."
```
- **Lessons Learned & Corrective Actions:**
 - Formats a corrective action checklist (e.g., - [] **[TODO] Correct underlying issue...**).
 - Lists lessons learned based on service propagation.
- **Export Options:**
 - **Markdown:** Persisted to **storage/rcas/{id}.md**.
 - **JSON:** Retrieved via **GET /rca/{id}/json**.
 - **PDF:** Handled by the frontend using **downloadPdfReport(...)** in [pdfGenerator.ts](#), which renders the document into a downloadable format.

11. Agent Observability

- **OpenTelemetry Implementation:** The application is instrumented using the OpenTelemetry SDK in [tracer.py](#).
- FastAPI is auto-instrumented via `FastAPIInstrumentor.instrument_app(app)` in `main.py`.
- Custom spans trace database calls, LLM queries, and Slack API deliveries using `tracer.start_as_current_span(...)`.
- ****Traces Collected**:** Spans trace ``neo4j.query.*``, ``redis.*`` (gets/sets), ``groq.get_reasoning``, and ``slack.post_message``.
- **Metrics Collected:** Defined in [metrics.py](#) and exposed via `GET /metrics`:
 - `incident_agent_heartbeat` (Watchdog heartbeat indicator)
 - `incidents_detected_total` (Tracks incident counts by severity/service)
 - `alert_deduplicated_total` (Tracks dropped alerts)
 - `llm_reasoning_latency_seconds` (Histogram tracking Groq call latencies)
 - `llm_token_usage_total` (Tracks token counts)
 - `slack_escalation_delivery_total` (Tracks Slack dispatches)
- **Watchdog Process:** Located in [watchdog.py](#). It runs a 30-second loop checking Redis and Neo4j connections. If healthy, it updates the `incident_agent_heartbeat` gauge.
- **Implementation Status:**
 - **Implemented:** FastAPI auto-instrumentation, custom span tracing, custom Prometheus metrics, and the watchdog process.
 - **Not Implemented:** Telemetry tracing is not used to dynamically discover the Neo4j topology. OpenTelemetry logs are not collected (only trace spans are exported).

12. Redis Usage

Redis serves as the central data store, message broker state checkpointer, and deduplication engine.

- **State Storage:** Incident payloads are saved under the key `incident:state:{id}` with a 24-hour TTL (`ex=86400`).
- **Incident Isolation:**
 - Active incidents are tracked using the set key `"incident:active_ids"`.
 - Incidents impacting specific services are indexed under `"service:active_incidents:{service}"` to enable fast lookups.
- **Caching:** Neo4j path searches and upstream/downstream lists are cached for 300 seconds using keys like `cache:neo4j:*` to reduce query load on the graph database.
- **Routing:** The channel routing configuration is stored in the hash key `"slack:channel_routing"`.
- **Deduplication:** Implemented in [redis_client.py](#) using a custom Bloom filter. It hashes incoming alerts (Name, Service, Severity, Instance) into 3 indices and checks rotating bitmaps `dedup:bloom:{epoch}` (where the epoch changes every 30 seconds). If the bits are already set, the alert is dropped as a duplicate.

13. API Inventory

Method	Endpoint	Purpose	Sample Response
POST	/alerts	Ingests webhook alerts from Prometheus or Grafana.	<pre>{ "status": "accepted", "processed_alerts_count": 1, "incidents_mapped": [{ "incident_id": "uuid", "action": "created" }] }</pre>
POST	/topology/load	Overwrites or seeds the Neo4j dependency graph.	<pre>{ "status": "success", "inserted_services": 5, "inserted_databases": 3, "inserted_dependencies": 7 }</pre>
GET	/topology/services/{name}	Retrieves upstream and downstream dependencies for a service.	<pre>{ "name": "auth-service", "type": "Service", "properties": {}, "upstreams": ["api-gateway"], "downstreams": [{ "type": "Database", "name": "user-db" }] }</pre>
GET	/topology/graph	Returns all nodes and edges in the Neo4j graph.	<pre>{ "nodes": [{ "id": "auth-service", "label": "Service", "properties": {} }], "edges": [{ "source": "api-gateway", "target": "auth-service", "properties": {} }] }</pre>
GET	/dashboard/metrics	Calculates SRE health and cost metrics.	<pre>{ "incidents_detected_per_hour": 1, "detection_accuracy": 97.8, "false_positive_rate": 0.05, "false_positive_estimated": true, "llm_call_volume": 4, "llm_cost": 0.08 }</pre>
POST	/incidents/{id}/resolve	Manually resolves an incident and compiles the post-mortem.	<pre>{ "id": "uuid", "state": "resolved", "services_affected": ["auth-service"], "rca_document_url": "/rca/uuid" }</pre>
GET	/incidents	Retrieves all incidents stored in Redis.	<pre>[{ "id": "uuid", "state": "open", "severity": "critical", "services_affected": ["auth-service"] }]</pre>

GET	/incidents/{id}	Retrieves a single incident from Redis by its ID.	{"id": "uuid", "state": "open", "severity": "critical", "services_affected": ["auth-service"]}
GET	/timeline/{id}	Retrieves the sorted timeline of events for an incident.	{"incident_id": "uuid", "timeline": [{"timestamp": "...", "event_type": "alert_triggered"}]}
GET	/rca/{id}	Retrieves the raw Markdown content of the RCA report.	Raw Markdown text representation.
PUT	/rca/{id}	Updates the RCA report content on disk and in Redis.	{"status": "success", "message": "RCA updated successfully."}
GET	/rca/{id}/export	Exports the RCA Markdown file as a downloadable payload.	Binary file transfer payload stream.
GET	/rca/{id}/json	Retrieves the structured JSON metadata for the RCA.	{"incident_id": "uuid", "title": "RCA Report", "markdown_content": "...", "resolved_at": "..."} }
GET	/incidents/{id}/channels	Returns available Slack channels for incident routing.	["#sre-alerts", "#prod-incidents"]
POST	/incidents/{id}/approve	Approves the incident and sends the Block Kit card to Slack.	{"id": "uuid", "state": "escalated", "approved_by": "operator"}
POST	/incidents/{id}/reject	Rejects the escalation, changing state to approval_rejected .	{"id": "uuid", "state": "approval_rejected", "rejected_by": "operator"}
POST	/slack/events	Slack webhook event receiver.	Passes request handling to AsyncSlackRequestHandler.

14. Database Inventory

1. Redis

Redis acts as a key-value store for application state, using key namespaces to prevent data overlap:

- **incident:state:{id}**: Holds the serialized JSON string of the **IncidentStateModel** (incidents expire after 24 hours).
- **incident:active_ids**: Set index containing UUIDs of all open, analyzing, or escalated incidents.
- **service:active_incidents:{service}**: Set index containing active incident IDs mapping to specific services.
- **dedup:bloom:{epoch}**: Bitmaps containing Bloom filter indices for alert deduplication (90-second expiration).
- **slack:channel_routing**: Hash mapping service names to target Slack channel names.
- **cache:neo4j:***: String values containing cached upstream/downstream details (5-minute TTL).
- **rca:{id}**: Holds the JSON representation of the post-mortem report (7-day TTL).

2. Neo4j

Neo4j stores the structural dependency graph:

- **Nodes:**
 - **Service** nodes: Contain metadata attributes (**name**, **language**, **version**, **updated_at**).
 - **Database** nodes: Contain metadata attributes (**name**, **type**, **updated_at**).
- **Relationships:**
 - **DEPENDS_ON** relationships: Connect Services and Databases, containing parameters (**protocol**, **p99_latency_threshold_ms**, **updated_at**).

15. Testing Verification

The test suite contains unit, integration, and E2E validation suites:

- **Unit Tests ([tests/unit/](#)):**

- [test_correlation.py](#): Verifies alert grouping, time-window logic, Neo4j dependency path routing, and multi-incident merging.
- [test_deduplication.py](#): Asserts Bloom filter indexes and rotates epoch bitmaps correctly.
- [test_groq.py](#): Asserts temperature configurations, JSON formatting options, and mock outputs.
- [test_neo4j.py](#): Tests Cypher executions, constraints, and upstream lookup queries.
- [test_observability.py](#): Verifies Prometheus metrics increment and watchdog health logging.
- [test_rca.py](#): Tests timeline sorting and post-mortem markdown template construction.
- [test_slack.py](#): Mocks Bolt app callbacks, ack hooks, and Block Kit builders.
- [test_telemetry.py](#): Simulates Prometheus metrics range calls and Loki log line queries.

- **Integration Tests ([tests/integration/](#)):**

- [test_alert_flow.py](#): Verifies alert ingestion endpoints, background tasks, and deduplication logic.
- [test_approval_flow.py](#): Asserts state updates for approvals/rejections and Slack dispatch calls.
- [test_chaos.py](#): Tests database failures and verifies safety fallback configurations.
- [test_deployment.py](#): Confirms operational health of external endpoints.
- [test_workflow.py](#): Validates LangGraph state machine execution, including checkpoint state storage.

- **E2E Validation Tests:**

- [dashboard.spec.ts](#): Uses Playwright to test the UI. It validates:
 - Login page layout.
 - Sidebar navigation.
 - Metric cards and charts.
 - Incident list search/filter functionality.
 - The incident resolution workflow.

- **Build Validation:**

- Verified using **tsc & vite build** on the frontend and **ruff check** on the backend to ensure zero compile errors or formatting warnings.

16. Requirement Compliance Matrix

Requirement	Implemented?	Code Evidence	Responsible File	Status
Deduplicate Alerts	■ Yes	Bitmaps with SHA-256 salts in <code>check_deduplicate</code>	redis_client.py	■ Fully Implemented
Group by Window	■ Yes	Sliding 300-second window in <code>correlate_alert</code>	correlation.py	■ Fully Implemented
Group by Dependency	■ Yes	Cypher queries tracing paths of length 1 or 2	neo4j_client.py	■ Fully Implemented
LLM Diagnostics	■ Yes	Invokes Llama 3.3 via Groq Chat Completions API	groq_client.py	■ Fully Implemented
Workflow Retries	■ Yes	Retries loop up to 3 times on schema validation errors	workflow.py	■ Fully Implemented
Manual Approval	■ Yes	UI approval screen routing to <code>/approve</code> endpoint	routes.py	■ Fully Implemented
Slack Escalations	■ Yes	Posts interactive Block Kit cards with Bolt SDK	slack_client.py	■ Fully Implemented
Action Timeline	■ Yes	Appends audit trails for automated and manual actions	rca_generator.py	■ Fully Implemented
Post-Mortem Gen	■ Yes	Generates Markdown reports and saves them to local storage	rca_generator.py	■ Fully Implemented
Auto Topology	■ No	Static loading; trace-based discovery is not implemented	main.py	■ Not Implemented
Traces / Metrics	■ Yes	OpenTelemetry auto-instrumentation & custom Prometheus metrics	main.py	■ Fully Implemented

17. Seminar Preparation Section

A. 5-Minute Project Elevator Pitch

"Our project implements an AI-Powered SRE Incident Management and Diagnostics Platform designed to solve two core challenges in modern microservices architectures: alert fatigue and high Mean Time To Repair (MTTR).

When a cascade failure occurs, alert systems flood SREs with duplicate alerts. Our platform solves this at the ingestion layer using Redis Bloom filters for deduplication and a correlation engine backed by Neo4j to group alerts based on service dependencies.

Once correlated, the platform launches a stateful LangGraph workflow to collect telemetry (Prometheus metrics and Loki logs) and run LLM-powered diagnostics using Groq Llama 3.3. This generates root-cause hypotheses, evidence, and remediation plans.

To ensure human oversight, the platform uses a manual-approval workflow. SREs review the AI diagnostics before sending an interactive Block Kit escalation card to Slack, where they can acknowledge or resolve the incident directly. Finally, the platform generates a comprehensive post-mortem report, reducing the time required to compile post-incident documentation."

B. 10-Minute Project Presentation Structure

1. **Slide 1: Title & Overview** — Present the system as a stateful, AI-powered diagnostic and escalation helper for SREs.
2. **Slide 2: Problem Statement** — Discuss alert storms, duplicate alerts, the challenges of manual root-cause analysis, and high MTTR.
3. **Slide 3: Architecture Overview** — Introduce the frontend, the FastAPI gateway, Redis for state, Neo4j for topology, LangGraph, and Slack.
4. **Slide 4: Alert Correlation** — Explain the 300-second window, direct service matching, and Cypher queries path matching in Neo4j.
5. **Slide 5: State Graph Workflow** — Detail the LangGraph workflow: data collection, Groq reasoning, and self-correction loops on JSON parsing errors.
6. **Slide 6: Slack Escalations** — Show how Slack interactive buttons map directly to FastAPI callback handlers to update incident states.
7. **Slide 7: Post-Mortem Generation** — Explain how the platform compiles timelines, hypotheses, and Groq summaries into Markdown reports.
8. **Slide 9: Gaps & Next Steps** — Address current limitations, such as using a static Neo4j topology instead of dynamic trace-based discovery.
9. **Slide 10: Conclusion & QA** — Summarize key achievements and invite questions.

C. Architecture Design Review

The backend is built around a single-process FastAPI application that coordinates backing databases and external APIs. OpenTelemetry middleware instruments incoming HTTP requests, recording trace spans that are exported via gRPC or HTTP OTLP processors.

The ingestion pipeline handles high alert volumes using an active-active Redis client. The Bloom filter uses bitwise operations on epoch-rotated bitmaps, allowing the system to drop duplicate alerts with minimal CPU overhead.

The core reasoning logic is decoupled into a LangGraph StateGraph. By binding a **RedisCheckpointSaver** to the graph, execution states are persisted across restarts. Telemetry data is queried in parallel using python's **asyncio.gather** client, reducing the time required to compile the context sent to Groq.

D. Common Viva Questions & Answers

Q1: How does the alert deduplication Bloom filter prevent memory leaks over time?

- **A:** It uses rotating epoch keys. The epoch is computed as `now // 30` (30-second windows). Bitmaps are written to the current epoch key with a 90-second Redis TTL. Old epoch keys automatically expire and are deleted by Redis, preventing memory leakage.

Q2: What happens if the Groq LLM API is rate-limited or fails during diagnostics?

- **A:** The Groq client is wrapped in a Tenacity retry block that catches `HTTPStatusError` (status code 429 or 5xx) and retries using exponential backoff. If it fails after 3 attempts, the LangGraph workflow falls back to a safe programmatic explanation.

Q3: How does the system handle concurrent updates if two operators click Acknowledge at the same time?

- **A:** Updates are processed by FastAPI endpoints that read the current state from Redis, update it, and write it back. To prevent race conditions in production, this should be updated to use Redis transaction watches or distributed locks.

Q4: How does the Neo4j correlation query prevent infinite loops on circular dependencies?

- **A:** The Cypher query specifies a maximum path depth of 2 (`-[:DEPENDS_ON*1..2]-`), which prevents infinite recursion and keeps query latency low.

Q5: Why did you choose LangGraph instead of a sequential API call script?

- **A:** LangGraph provides state management and supports conditional routing (e.g., looping back to the LLM on validation errors). It also allows us to persist execution states, making it easier to resume interrupted runs.

E. Potential Weaknesses Identified by Examiners

1. **Static Topology:** The Neo4j graph is seeded from a static JSON file rather than dynamically updated from active service telemetry.
2. **Local Disk Storage:** Post-mortem reports are saved as Markdown files on the local filesystem, which is not suitable for horizontally scaled deployments.
3. **No Authentication on Backend:** The FastAPI routes do not require authentication or authorization tokens, exposing endpoints to unauthorized requests.
4. **Race Conditions:** State updates in Redis are not protected by locks, which could lead to conflicts if multiple operators update the same incident concurrently.

18. Final Assessment

- **Overall Completion Percentage: 95%**
- **Ingestion, Deduplication, & Correlation:** 100% complete (fully implemented in Redis and Neo4j).
- **LangGraph AI Reasoning:** 100% complete (with self-correction and fallback handling).
- **Operator Escalation & Slack Integration:** 100% complete (using Bolt SDK interactive buttons).
- **Post-Mortem Generation:** 100% complete (including exports and checklist tracking).
- **Observability & Health Checks:** 90% complete (watchdog, metrics, and tracing are active, but trace-based topology discovery is missing).
- **Strengths:**
 - Robust deduplication using epoch-rotated Bloom filters.
 - Stateful, self-correcting diagnostics using LangGraph.
 - Interactive Slack integrations using Bolt SDK cards.
 - High-quality UI with real-time updates and interactive graphs.
- **Missing Features / Gaps:**
 - Dynamic, trace-based topology discovery in Neo4j.
 - Cloud-backed storage (e.g., AWS S3) for compiled post-mortem reports.
 - Authentication and role-based access controls (RBAC) on backend API routes.
- **Production Readiness Assessment: Nearly Ready (Requires hardening).** The core application logic is robust and well-tested. However, before deploying to production, security must be improved by adding authentication middleware, and storage should be migrated to cloud-backed object storage.